

September 5, 2026 at 13:19

1. Introduction. Exercise 7.2.2.1–55 is about how few clues a sudoku puzzle can have. It picks two of the highly symmetrical squares displayed in (28) and asks, for each of them, how small a set of given cells can force it as the only answer.

The answer settles both. For (28a) eighteen clues are necessary and eighteen suffice; a puzzle is printed. For (28b) eighteen are again necessary but not sufficient, nineteen are not sufficient either, and twenty are; another puzzle is printed. Knuth reached the negative half with a SAT solver, and reports that ruling out nineteen clues took 177 megamems.

This program works through all of it without a SAT solver. The lower bound comes out of the same observation the exercise’s hint makes, and the negative results come out of a search that the observation makes small—small enough that ruling out eighteen clues takes a tenth of a second rather than a solver run.

```
package main
import (
    "flag"
    "fmt"
    "time"
)
⟨ Declarations 4 ⟩
⟨ Functions 6 ⟩
func main() {
    ⟨ Read the command line 2 ⟩
    ⟨ Do what the mode asks 3 ⟩
}
```

2. Six modes, one per claim to be checked. The squares mode looks at (28) itself; puzzles reads back the two puzzles the answer prints; groups checks the argument behind the bound; sets enumerates the sets that make the argument work; clues does the real search; and isotopy carries the whole construction from (28b) to (28c).

⟨ Read the command line 2 ⟩ ≡

```
mode := flag.String("mode", "squares", "squares, puzzles, groups, sets, clues, or isotopy")
which := flag.String("g", "b", "which square of (28)")
size := flag.Int("d", 6, "largest unavoidable set to look for")
clues := flag.Int("n", 18, "how many clues to search for")
list := flag.Bool("v", false, "print every puzzle found")
flag.Parse()
verbose = *list
```

This code is used in section 1.

3. $\langle$ Do what the mode asks 3 $\rangle \equiv$

```

switch *mode {
case "squares":
    showSquares()
case "puzzles":
    checkPuzzle("the_18-clue_puzzle_printed_for_(28a)", "a", puz28a)
    checkPuzzle("the_20-clue_puzzle_printed_for_(28b)", "b", puz28b)
case "groups":
    checkGroups("(28" + *which + ")", book[*which])
case "sets":
    reportSets("(28" + *which + ")", book[*which], *size)
case "clues":
    search("(28" + *which + ")", book[*which], *size, *clues)
case "isotopy":
    checkIsotopy()
default:
    fmt.Println("unknown_mode")
}

```

This code is used in section 1.

4. $\langle$ Declarations 4 $\rangle \equiv$

```

var verbose bool

```

This code is also defined in sections 5, 8, 15, 18.

This code is used in section 1.

5. The three squares. Here are the squares of (28), copied from the page. They are worth copying rather than computing, because answer 43 gives closed forms for all three and the two readings can then be played off against each other: if my transcription were wrong the formulas would disagree with it.

(Declarations 4) +≡

```
var book = map[string][9]string{
  "a": {
    "123456789", "456789123", "789123456",
    "234567891", "567891234", "891234567",
    "345678912", "678912345", "912345678",
  },
  "b": {
    "123456789", "456789123", "789123456",
    "231564897", "564897231", "897231564",
    "312645978", "645978312", "978312645",
  },
  "c": {
    "123456789", "564897231", "978312645",
    "645978312", "789123456", "231564897",
    "897231564", "312645978", "456789123",
  },
}
```

6. Writing $i = (i_1 i_0)_3$ and $j = (j_1 j_0)_3$, answer 43 says that the three squares, less one, are the multiplication tables

$$\begin{aligned} a'_{ij} &= ((i_0 i_1)_3 + j) \bmod 9, & b'_{ij} &= ((i_0 + j_1) \bmod 3, (i_1 + j_0) \bmod 3)_3, \\ c'_{ij} &= ((i_0 + i_1 + j_1) \bmod 3, (i_0 - i_1 + j_0) \bmod 3)_3. \end{aligned}$$

⟨ Functions 6 ⟩ ≡

```

func formula(which string) [9]string {
  var g [9]string
  for i := 0; i < 9; i++ {
    i0, i1 := i % 3, i / 3
    row := make([]byte, 9)
    for j := 0; j < 9; j++ {
      j0, j1 := j % 3, j / 3
      var v int
      switch which {
        case "a":
          v = (3 * i0 + i1 + j) % 9
        case "b":
          v = 3 * ((i0 + j1) % 3) + (i1 + j0) % 3
        case "c":
          v = 3 * ((i0 + i1 + j1) % 3) + ((i0 - i1 + j0) % 3 + 3) % 3
      }
      row[j] = byte('1' + v)
    }
    g[i] = string(row)
  }
  return g
}

```

This code is also defined in sections 7, 9, 10, 14, 16, 19, 20, 25, 26, 28, 29, 30, 34, 36, 37, 41, 42, 43.

This code is used in section 1.

7. $\langle \text{Functions 6} \rangle + \equiv$

```

func valid(g [9]string) bool {
    for i := 0; i < 9; i++ {
        var r, c, b [10]int
        for j := 0; j < 9; j++ {
            r[g[i][j] - '0']++
            c[g[j][i] - '0']++
            b[g[3 * (i/3) + j/3][3 * (i % 3) + j % 3] - '0']++
        }
        for d := 1; d ≤ 9; d++ {
            if r[d] ≠ 1 ∨ c[d] ≠ 1 ∨ b[d] ≠ 1 {
                return false
            }
        }
    }
    return true
}

func showSquares() {
    for _, w := range []string { "a", "b", "c" }{
        fmt.Printf("(28%s): a valid sudoku square %v, matches answer 43's formula %v\n",
            w, valid(book[w]), book[w] ≡ formula(w))
    }
}

```

8. Filling in a sudoku. A puzzle is eighty-one characters with a dot wherever a cell is blank. Counting its solutions is the innermost loop of everything that follows, so it branches on the cell with the fewest candidates and stops as soon as the caller has seen enough of them.

⟨Declarations 4⟩ +≡

```
type puzzle [81]byte
var boxOf [81]int
```

9. ⟨Functions 6⟩ +≡

```
func init() {
    for c := 0; c < 81; c++ {
        boxOf[c] = 3 * (c/27) + (c % 9)/3
    }
}

func fromGrid(g [9]string) puzzle {
    var p puzzle
    for i := 0; i < 9; i++ {
        for j := 0; j < 9; j++ {
            p[9 * i + j] = g[i][j]
        }
    }
    return p
}

func (p puzzle) clues() int {
    n := 0
    for _, c := range p {
        if c ≠ '.' {
            n++
        }
    }
    return n
}

func (p puzzle) String() string {
    s := ""
    for i := 0; i < 9; i++ {
        s += "UUUU" + string(p[9 * i:9 * i + 9]) + "\n"
    }
    return s
}
```

10. $\langle \text{Functions 6} \rangle + \equiv$

```

func (p puzzle) count(limit int) int {
  var row, col, box [9]uint16
   $\langle \text{Note the digits the clues use up 11} \rangle$ 
  n := 0
  var rec func()
  rec = func() {
     $\langle \text{Pick the cell with fewest candidates, or report a solution 12} \rangle$ 
     $\langle \text{Try each candidate there 13} \rangle$ 
  }
  rec()
  return n
}

```

11. $\langle \text{Note the digits the clues use up 11} \rangle \equiv$

```

for c := 0; c < 81; c++ {
  if p[c]  $\equiv$  ' .' {
    continue
  }
  b := uint16(1)  $\ll$  (p[c] - '1')
  row[c/9] |= b
  col[c%9] |= b
  box[boxOf[c]] |= b
}

```

This code is used in section 10.

12. $\langle \text{Pick the cell with fewest candidates, or report a solution 12} \rangle \equiv$

```

if n  $\geq$  limit {
  return
}
best, mask, fewest := -1, uint16(0), 10
for c := 0; c < 81; c++ {
  if p[c]  $\neq$  ' .' {
    continue
  }
  m :=  $\oplus$ (row[c/9] | col[c%9] | box[boxOf[c]]) & #1ff
  k := popcount(m)
  if k < fewest {
    best, mask, fewest = c, m, k
    if k  $\leq$  1 {
      break
    }
  }
}
if best < 0 {
  n++
  return
}

```

This code is used in section 10.

13. $\langle$ Try each candidate there 13 $\rangle \equiv$

```

for  $d := 0; d < 9; d++$  {
  if  $mask \& (1 \ll d) \equiv 0$  {
    continue
  }
   $b := \text{uint16}(1) \ll d$ 
   $p[best] = \text{byte}('1' + d)$ 
   $row[best/9] |= b$ 
   $col[best \% 9] |= b$ 
   $box[boxOf[best]] |= b$ 
   $rec()$ 
   $p[best] = '.'$ 
   $row[best/9] \oslash= b$ 
   $col[best \% 9] \oslash= b$ 
   $box[boxOf[best]] \oslash= b$ 
  if  $n \geq limit$  {
    return
  }
}

```

This code is used in section 10.

14. 〈Functions 6〉 +≡

```

func popcount(m uint16) int {
    n := 0
    for; m ≠ 0; m &= m - 1 {
        n ++
    }
    return n
}

// keep blanks every cell of a square except the ones named.
func keep(g [9]string, cells []int) puzzle {
    full := fromGrid(g)
    var p puzzle
    for i := range p {
        p[i] = '.'
    }
    for r, c := range cells {
        p[c] = full[c]
    }
    return p
}

func cellsOf(rows [9]string) []int {
    var out []int
    for i := 0; i < 9; i ++ {
        for j := 0; j < 9; j ++ {
            if rows[i][j] ≠ '.' {
                out = append(out, 9 * i + j)
            }
        }
    }
    return out
}

```

15. The two printed puzzles. Both are read back the same way: the clues must agree with the square they claim to determine, and the puzzle must have exactly one solution. While we are here it costs nothing to ask whether any clue could be spared.

⟨Declarations 4⟩ +≡

```
var puz28a = [9]string{
    "123.....", "...789...", ".....",
    "234.....", ".....", ".....567",
    "..567....", ".....", "91.....8",
}
var puz28b = [9]string{
    "1...6.8.", ".5....1..", "...9...4..",
    ".3.5....7", "....9....", "8....1...",
    ".....7.", "64...8..2", "...3....5",
}
```

16. ⟨Functions 6⟩ +≡

```
func checkPuzzle(name, which string, rows [9]string) {
    cs := cellsOf(rows)
    g := book[which]
    agree := true
    for i := 0; i < 9; i++ {
        for j := 0; j < 9; j++ {
            if rows[i][j] ≠ '.' ∧ rows[i][j] ≠ g[i][j] {
                agree = false
            }
        }
    }
    p := keep(g, cs)
    fmt.Printf("%s: %d clues, all agreeing with the square %v, solutions %d\n",
        name, p.clues(), agree, p.count(2))
    ⟨Count the clues that could be dropped 17⟩
    ⟨Count the clues in each group 33⟩
}
```

17. ⟨Count the clues that could be dropped 17⟩ ≡

```
spare := 0
for _, drop := range cs {
    var rest []int
    for _, c := range cs {
        if c ≠ drop {
            rest = append(rest, c)
        }
    }
    if keep(g, rest).count(2) ≡ 1 {
        spare++
    }
}
fmt.Printf("clues that could be dropped: %d\n", spare)
```

This code is used in section 16.

18. Unavoidable sets. Here is the idea the whole exercise turns on. Call a set of cells *unavoidable* if the digits in it can be rearranged to give another solution. Every clue set must meet every unavoidable set: a clue set that missed one would leave both rearrangements open, and the puzzle would have two answers. So the smallest number of clues is the size of the smallest hitting set of the unavoidable sets, and nothing else.

Only the minimal ones matter, since a set that properly contains an unavoidable set is met whenever that one is. Minimality is also what makes them findable. Walking over every grid within Hamming distance 18 of a given one is hopeless, but if the sets are found in order of size, then a partial difference that already covers a known set can be abandoned at once: whatever it grows into will not be minimal.

⟨Declarations 4⟩ +≡

```
type bits [2]uint64
```

19. ⟨Functions 6⟩ +≡

```
func (s *bits) set(c int) { s[c/64] |= 1 << uint(c % 64) }
func (s *bits) clear(c int) { s[c/64] &= ^ (1 << uint(c % 64)) }
func (s bits) has(c int) bool {
    return s[c/64] & (1 << uint(c % 64)) != 0
}
func (s bits) covers(t bits) bool {
    return s[0] & t[0] == t[0] & s[1] & t[1] == t[1]
}
func (s bits) meets(t bits) bool {
    return s[0] & t[0] != 0 & s[1] & t[1] != 0
}
func (s bits) size() int {
    n := 0
    for _, w := range s {
        for; w != 0; w &= w - 1 {
            n++
        }
    }
    return n
}
func (s bits) cells() []int {
    var out []int
    for c := 0; c < 81; c++ {
        if s.has(c) {
            out = append(out, c)
        }
    }
    return out
}
```

20. $\langle \text{Functions 6} \rangle + \equiv$

```

func minimalSets(g [9]string, maxSize int, show bool) []bits {
    full := fromGrid(g)
    var found []bits
    endsAt := make([]bits, 81)
    for size := 4; size ≤ maxSize; size++ {
        t0, n0 := time.Now(), len(found)
         $\langle \text{Look for the minimal sets of this size 21} \rangle$ 
         $\langle \text{File the new sets under their last cell 24} \rangle$ 
        if show {
            fmt.Printf("    size_%2d: %d minimal sets (%s)\n",
                size, len(found) - n0, time.Since(t0).Round(time.Millisecond))
        }
    }
    return found
}

```

21. The walk fills the grid cell by cell, keeping count of how many cells differ from the square we started with. Two things hold it back. A difference that already covers a set found earlier is dropped, since it cannot be minimal. And a column that has been changed exactly once so far must be changed at least once more, because a column with a single alteration is no longer a permutation of the nine digits; counting those columns gives a lower bound on the changes still to come.

$\langle \text{Look for the minimal sets of this size 21} \rangle \equiv$

```

var row, col, box [9]uint16
var colDiff [9]int
var diff bits
nd := 0
var rec func(c int)
rec = func(c int) {
    if c ≡ 81 {
        if nd ≡ size {
            found = append(found, diff)
        }
        return
    }
     $\langle \text{Give up if the columns already promise too much 22} \rangle$ 
     $\langle \text{Try every digit that fits in this cell 23} \rangle$ 
}
rec(0)

```

This code is used in section 20.

22. $\langle$ Give up if the columns already promise too much 22 $\rangle \equiv$

```

if  $c \% 9 \equiv 0$  {
   $lb := 0$ 
  for  $j := 0; j < 9; j++$  {
    if  $colDiff[j] \equiv 1$  {
       $lb++$ 
    }
  }
  if  $nd + lb > size$  {
    return
  }
}

```

This code is used in section 21.

23. $\langle$ Try every digit that fits in this cell 23 $\rangle \equiv$

```

 $m := \oplus(row[c/9] \mid col[c \% 9] \mid box[boxOf[c]]) \& \#1ff$ 
for  $d := 0; d < 9; d++$  {
  if  $m \& (1 \ll d) \equiv 0$  {
    continue
  }
   $v := \text{byte}('1' + d)$ 
   $changed := v \neq full[c]$ 
  if  $changed \wedge nd \equiv size$  {
    continue
  }
  if  $changed$  {
     $diff.set(c)$ 
    if  $covered(diff, endsAt[c])$  {
       $diff.clear(c)$ 
      continue
    }
     $nd++$ 
     $colDiff[c \% 9]++$ 
  }
   $b := \text{uint16}(1) \ll d$ 
   $row[c/9] \mid= b$ 
   $col[c \% 9] \mid= b$ 
   $box[boxOf[c]] \mid= b$ 
   $rec(c + 1)$ 
   $row[c/9] \oslash= b$ 
   $col[c \% 9] \oslash= b$ 
   $box[boxOf[c]] \oslash= b$ 
  if  $changed$  {
     $diff.clear(c)$ 
     $nd--$ 
     $colDiff[c \% 9]--$ 
  }
}

```

This code is used in section 21.

24. Cells are filled in increasing order, so a set first lies inside the difference exactly when its last cell is added. Filing the sets that way turns the containment test into a glance at a short list.

⟨File the new sets under their last cell 24⟩ ≡

```

for _, s := range found[n0:] {
    cs := s.cells()
    last := cs[len(cs) - 1]
    endsAt[last] = append(endsAt[last], s)
}

```

This code is used in section 20.

25. ⟨Functions 6⟩ +≡

```

func covered(diff bits, known []bits) bool {
    for _, k := range known {
        if diff.covers(k) {
            return true
        }
    }
    return false
}

```

26. Nothing here is taken on trust. A set found by the walk is tested on its own terms: blank it, give every other cell, and there had better be more than one way to fill it back in—and handing any single cell back had better spoil that.

⟨Functions 6⟩ +≡

```

func reportSets(name string, g [9]string, maxSize int) {
    fmt.Printf("%s:\n", name)
    sets := minimalSets(g, maxSize, true)
    full := fromGrid(g)
    bad, notMinimal := 0, 0
    completions := map[int]int{ }
    for _, s := range sets {
        ⟨Test this set the slow, honest way 27⟩
    }
    fmt.Printf("%%d minimal sets of at most %%d cells\n", len(sets), maxSize)
    fmt.Printf("%%all genuinely unavoidable: %%v, %%all genuinely minimal: %%v\n",
        bad == 0, notMinimal == 0)
    fmt.Printf("%%completions when a set is blanked:")
    for k := 2; k ≤ 9; k++ {
        if completions[k] > 0 {
            fmt.Printf("%%d->%%d", k, completions[k])
        }
    }
    fmt.Println()
}

```

27. $\langle$ Test this set the slow, honest way 27 $\rangle \equiv$

```

var p puzzle
for c := 0; c < 81; c ++ {
  if s.has(c) {
    p[c] = '.'
  } else {
    p[c] = full[c]
  }
}
n := p.count(9)
completions[n] ++
if n < 2 {
  bad ++
}
for _, c := range s.cells() {
  q := p
  q[c] = full[c]
  if q.count(2) > 1 {
    notMinimal ++
    break
  }
}

```

This code is used in section 26.

28. The nine groups. The hint says that at least two of the nine appearances of $\{1, 4, 7\}$ in the top three rows must be clues, and the answer adds that the whole diagram splits into nine disjoint sets of nine with the same property. That is the bound: two clues apiece, nine sets, eighteen clues.

The nine cells of the hint turn out to be the first column of each box in the top band, and the sets in general are one band of rows crossed with one column of each box.

(Functions 6) +≡

```
func group(band, class int) []int {
  var out []int
  for i := 3 * band; i < 3 * band + 3; i++ {
    for j := 0; j < 9; j++ {
      if j % 3 == class {
        out = append(out, 9 * i + j)
      }
    }
  }
  return out
}
```

29. The property is checked one cell at a time: hand back a single cell of the set, blank the other eight, give everything else, and count.

(Functions 6) +≡

```
func twoAreNeeded(g [9]string, set []int)(bool, int) {
  full := fromGrid(g)
  in := map[int]bool{}
  for _, c := range set {
    in[c] = true
  }
  fewest := 99
  for _, keepCell := range set {
    var p puzzle
    for c := 0; c < 81; c++ {
      if in[c] ^ c != keepCell {
        p[c] = '.'
      } else {
        p[c] = full[c]
      }
    }
    n := p.count(9)
    if n < fewest {
      fewest = n
    }
    if n < 2 {
      return false, n
    }
  }
  return true, fewest
}
```

30. $\langle \text{Functions } 6 \rangle + \equiv$

```

func checkGroups(name string, g [9]string) {
    fmt.Printf("%s:\n", name)
     $\langle \text{Check the nine cells the hint names } 31 \rangle$ 
     $\langle \text{Check the nine groups that partition the square } 32 \rangle$ 
}

```

31. $\langle \text{Check the nine cells the hint names } 31 \rangle \equiv$

```

var hint []int
for i := 0; i < 3; i++ {
    for j := 0; j < 9; j++ {
        if g[i][j]  $\equiv$  '1'  $\vee$  g[i][j]  $\equiv$  '4'  $\vee$  g[i][j]  $\equiv$  '7' {
            hint = append(hint, 9 * i + j)
        }
    }
}
ok, fewest := twoAreNeeded(g, hint)
fmt.Printf("the nine appearances of 1, 4, 7 in the top three rows: %v\n", hint)
fmt.Printf("one clue among them is never enough: %v (fewest completions %d)\n", ok,
    fewest)

```

This code is used in section 30.

32. $\langle \text{Check the nine groups that partition the square } 32 \rangle \equiv$

```

seen := map[int]bool{ }
all := true
for band := 0; band < 3; band++ {
    for class := 0; class < 3; class++ {
        for _, c := range group(band, class) {
            seen[c] = true
        }
        good, _ := twoAreNeeded(g, group(band, class))
        if  $\neg$ good {
            all = false
        }
    }
}
fmt.Printf("nine disjoint groups covering %d cells, each needing two clues: %v\n",
    len(seen), all  $\wedge$  len(seen)  $\equiv$  81)

```

This code is used in section 30.

33. $\langle$ Count the clues in each group 33 $\rangle \equiv$

```

in := map[int]bool{}
for _, c := range cs {
    in[c] = true
}
fmt.Printf("  clues per group:")
for band := 0; band < 3; band++ {
    for class := 0; class < 3; class++ {
        n := 0
        for _, c := range group(band, class) {
            if in[c] {
                n++
            }
        }
        fmt.Printf("  %d", n)
    }
}
fmt.Println()

```

This code is used in section 16.

34. Every puzzle of a given size. Now the bound turns into a search. The groups partition the square and each of them needs two clues, so a puzzle with eighteen clues has exactly two in every group and not one to spare; a puzzle with nineteen has three in one group and two in the rest; and so on. Within a group, a pair of cells has to meet every unavoidable set that lies wholly inside the group, and only nine of the thirty-six pairs do. Eighteen clues therefore means $9^9 = 387,420,489$ candidates—a huge number, but a finite and very structured one.

What cuts it down is the unavoidable sets that straddle two groups. Each of them can be tested as soon as the last group it touches has been settled, so the tree is pruned as it is built. Square (28a) has no straddling sets of six cells at all; (28b) has 81 of them, and that difference is the whole distance between eighteen clues and twenty.

⟨ Functions 6 ⟩ +≡

```
func search(name string, g [9]string, maxSet, want int) {
    t0 := time.Now()
    sets := minimalSets(g, maxSet, false)
    var groups [9][]int
    var gmask [9]bits
    for band := 0; band < 3; band++ {
        for class := 0; class < 3; class++ {
            k := 3 * band + class
            groups[k] = group(band, class)
            for _, c := range groups[k] {
                gmask[k].set(c)
            }
        }
    }
    ⟨ File each set under the last group it touches 35 ⟩
    ⟨ Run over every way of placing the clues 38 ⟩
    fmt.Printf("%s with %d clues: %d candidates survived, %d puzzles, %s\n",
        name, want, candidates, puzzles, time.Since(t0).Round(time.Millisecond))
}
```

35. ⟨ File each set under the last group it touches 35 ⟩ ≡

```
var endsAt [9][]bits
for _, s := range sets {
    last := 0
    for k := 0; k < 9; k++ {
        if gmask[k].meets(s) {
            last = k
        }
    }
    endsAt[last] = append(endsAt[last], s)
}
```

This code is used in section 34.

36. The subsets of a group that will do: those meeting every unavoidable set inside it.

(Functions 6) +≡

```

func usable(cells []int, inside []bits, size int) []bits {
  var out []bits
  var pick func(start int, chosen bits, left int)
  pick = func(start int, chosen bits, left int) {
    if left ≡ 0 {
      for s := range inside {
        if ¬chosen.meets(s) {
          return
        }
      }
      out = append(out, chosen)
      return
    }
    for i := start; i ≤ len(cells) − left; i++ {
      t := chosen
      t.set(cells[i])
      pick(i + 1, t, left − 1)
    }
  }
  pick(0, bits{}, size)
  return out
}

```

37. Clues beyond the eighteen may fall anywhere, and they need not fall together: twenty clues can be four in one group or three in each of two, and the answer's own twenty-clue puzzle is of the second kind. So every way of handing out the surplus is tried.

(Functions 6) +≡

```

func shares(extra int) [][9]int {
  var out [][9]int
  var cur [9]int
  var rec func(k, left int)
  rec = func(k, left int) {
    if k ≡ 9 {
      if left ≡ 0 {
        out = append(out, cur)
      }
      return
    }
    for v := 0; v ≤ left; v++ {
      cur[k] = v
      rec(k + 1, left − v)
    }
    cur[k] = 0
  }
  rec(0, extra)
  return out
}

```

38. $\langle$ Run over every way of placing the clues 38 $\rangle \equiv$

```

candidates, puzzles := 0, 0
full := fromGrid(g)
for  $\_, share := \text{range } shares(want - 18)$  {
  var choice [9][bits]
  for  $k := 0; k < 9; k++$  {
    var inside [bits]
    for  $\_, s := \text{range } sets$  {
      if gmask[ $k$ ].covers( $s$ ) {
        inside = append(inside,  $s$ )
      }
    }
    choice[ $k$ ] = usable(groups[ $k$ ], inside,  $2 + share[k]$ )
  }
   $\langle$  Walk the groups, pruning as each one is settled 39  $\rangle$ 
}

```

This code is used in section 34.

39. $\langle$ Walk the groups, pruning as each one is settled 39 $\rangle \equiv$

```

var chosen bits
var rec func( $k$  int)
rec = func( $k$  int) {
  if  $k \equiv 9$  {
     $\langle$  Test this candidate for good 40  $\rangle$ 
    return
  }
  for  $\_, pick := \text{range } choice[k]$  {
    save := chosen
    chosen[0] |= pick[0]
    chosen[1] |= pick[1]
    ok := true
    for  $\_, s := \text{range } endsAt[k]$  {
      if  $\neg chosen.meets(s)$  {
        ok = false
        break
      }
    }
    if ok {
      rec( $k + 1$ )
    }
    chosen = save
  }
}
rec(0)

```

This code is used in section 38.

40. A candidate that has met every unavoidable set the program knows about still has to be put to the real test, which is simply to solve it.

⟨Test this candidate for good 40⟩ ≡

```

candidates ++
var p puzzle
for i := 0; i < 81; i ++ {
  if chosen.has(i) {
    p[i] = full[i]
  } else {
    p[i] = ' .'
  }
}
if p.count(2) ≡ 1 {
  puzzles ++
  if verbose {
    fmt.Print(p.String())
  }
}

```

This code is used in section 39.

41. The isotopism. The answer ends by saying that the constructions for (28b) apply to (28c) as well, through the isotopism of answer 43. That answer names the map: writing an index as $(x_1x_0)_3$, the permutation π sends it to $(x_1, (x_0 + x_1) \bmod 3)_3$, and then $c'_{ij} = b'_{(i\pi)(j\pi^{-})}\pi - \pi$ on the row, its inverse on the column, and π again on the digit.

Rather than trust a reading of that formula, the program looks for the isotopism itself, among the permutations a sudoku allows: the three bands may be permuted and so may the three indices inside each band.

⟨ Functions 6 ⟩ +≡

```

func pi(x int) int {
  x1, x0 := x/3, x % 3
  return 3 * x1 + (x0 + x1) % 3
}

func bandPerms() [][9]int {
  three := [][3]int{ {0, 1, 2}, {0, 2, 1}, {1, 0, 2}, {1, 2, 0}, {2, 0, 1}, {2, 1, 0} }
  var out [][9]int
  for _, outer := range three {
    for _, a := range three {
      for _, b := range three {
        for _, c := range three {
          inner := [3][3]int{ a, b, c }
          var p [9]int
          for band := 0; band < 3; band++ {
            for k := 0; k < 3; k++ {
              p[3 * band + k] = 3 * outer[band] + inner[band][k]
            }
          }
          out = append(out, p)
        }
      }
    }
  }
  return out
}

```

42. $\langle$ Functions 6 $\rangle + \equiv$

```

func findIsotopy(b, c [9]string)(rho, gam [9]int, sig [9]int, ok bool) {
    perms := bandPerms()
    for r, r := range perms {
        for g, g := range perms {
            var s [9]int
            for i := range s {
                s[i] = -1
            }
            good := true
            for i := 0; i < 9 ∧ good; i++ {
                for j := 0; j < 9; j++ {
                    from, to := int(b[i][j] - '1'), int(c[r][i][j] - '1')
                    if s[from] ≡ -1 {
                        s[from] = to
                    } else if s[from] ≠ to {
                        good = false
                        break
                    }
                }
            }
            if good {
                return r, g, s, true
            }
        }
    }
    return rho, gam, sig, false
}

```

43. $\langle$ Functions 6 $\rangle + \equiv$

```

func checkIsotopy() {
    b, c := book["b"], book["c"]
    r, g, s, ok := findIsotopy(b, c)
    fmt.Printf("(28c) is an isotope of (28b): %v\n", ok)
    if ¬ok {
        return
    }
    fmt.Printf("rows %v, columns %v, digits %v\n", r, g, s)
     $\langle$  Check the formula exactly as answer 43 prints it 44  $\rangle$ 
     $\langle$  Carry the groups and the puzzle across 45  $\rangle$ 
}

```

44. $\langle$ Check the formula exactly as answer 43 prints it 44 $\rangle \equiv$

```

var back [9]int
for x := 0; x < 9; x++ {
  back[pi(x)] = x
}
printed := true
for i := 0; i < 9; i++ {
  for j := 0; j < 9; j++ {
    if c[i][j] ≠ byte('1' + pi(int(b[pi(i)][back[j]] - '1')) {
      printed = false
    }
  }
}
fmt.Printf("the formula printed in answer 43 holds exactly: %v\n", printed)

```

This code is used in section 43.

45. $\langle$ Carry the groups and the puzzle across 45 $\rangle \equiv$

```

carry := func(cells []int) []int {
  var out []int
  for _, x := range cells {
    out = append(out, 9 * r[x/9] + g[x % 9])
  }
  return out
}
seen := map[int]bool{}
all := true
for band := 0; band < 3; band++ {
  for class := 0; class < 3; class++ {
    set := carry(group(band, class))
    for _, x := range set {
      seen[x] = true
    }
    if good, _ := twoAreNeeded(c, set); ¬good {
      all = false
    }
  }
}
}
fmt.Printf("the nine groups carry over, covering %d cells, each needing two: %v\n",
  len(seen), all ∧ len(seen) == 81)
q := keep(c, carry(cellsOf(puz28b)))
fmt.Printf("the 20-clue puzzle carries over: %d clues, solutions %d\n",
  q.clues(), q.count(2))
fmt.Print(q.String())

```

This code is used in section 43.

46. Index.

- a*: [41](#).
agree: [16](#).
all: [32](#), [45](#).
append: [14](#), [17](#), [19](#), [21](#), [24](#), [28](#), [31](#), [35](#), [36](#), [37](#), [38](#), [41](#), [45](#).
b: [7](#), [11](#), [13](#), [23](#), [41](#), [42](#), [43](#), [44](#).
back: [44](#).
bad: [26](#), [27](#).
band: [28](#), [32](#), [33](#), [34](#), [41](#), [45](#).
bandPerms: [41](#), [42](#).
best: [12](#), [13](#).
bits: [18](#), [19](#), [20](#), [21](#), [25](#), [34](#), [35](#), [36](#), [38](#), [39](#).
book: [3](#), [5](#), [7](#), [16](#), [43](#).
Bool: [2](#).
bool: [4](#), [7](#), [19](#), [20](#), [25](#), [29](#), [32](#), [33](#), [42](#), [45](#).
box: [10](#), [11](#), [12](#), [13](#), [21](#), [23](#).
boxOf: [8](#), [9](#), [11](#), [12](#), [13](#), [23](#).
byte: [6](#), [8](#), [13](#), [23](#), [44](#).
c: [7](#), [9](#), [11](#), [12](#), [14](#), [17](#), [19](#), [21](#), [22](#), [23](#), [27](#), [29](#), [32](#), [33](#), [34](#), [41](#), [42](#), [43](#), [44](#), [45](#).
candidates: [34](#), [38](#), [40](#).
carry: [45](#).
cells: [14](#), [19](#), [24](#), [27](#), [36](#), [45](#).
cellsOf: [14](#), [16](#), [45](#).
changed: [23](#).
checkGroups: [3](#), [30](#).
checkIsotopy: [3](#), [43](#).
checkPuzzle: [3](#), [16](#).
choice: [38](#), [39](#).
chosen: [36](#), [39](#), [40](#).
class: [28](#), [32](#), [33](#), [34](#), [45](#).
clear: [19](#), [23](#).
clues: [2](#), [3](#), [9](#), [16](#), [45](#).
col: [10](#), [11](#), [12](#), [13](#), [21](#), [23](#).
colDiff: [21](#), [22](#), [23](#).
completions: [26](#), [27](#).
count: [10](#), [16](#), [17](#), [27](#), [29](#), [40](#), [45](#).
covered: [23](#), [25](#).
covers: [19](#), [25](#), [38](#).
cs: [16](#), [17](#), [24](#), [33](#).
cur: [37](#).
d: [7](#), [13](#), [23](#).
diff: [21](#), [23](#), [25](#).
drop: [17](#).
endsAt: [20](#), [23](#), [24](#), [35](#), [39](#).
extra: [37](#).
fewest: [12](#), [29](#), [31](#).
findIsotopy: [42](#), [43](#).
flag: [2](#).
fmt: [3](#), [7](#), [16](#), [17](#), [20](#), [26](#), [30](#), [31](#), [32](#), [33](#), [34](#), [40](#), [43](#), [44](#), [45](#).
formula: [6](#), [7](#).
found: [20](#), [21](#), [24](#).
from: [42](#).
fromGrid: [9](#), [14](#), [20](#), [26](#), [29](#), [38](#).
full: [14](#), [20](#), [23](#), [26](#), [27](#), [29](#), [38](#), [40](#).
g: [6](#), [7](#), [9](#), [14](#), [16](#), [17](#), [20](#), [26](#), [29](#), [30](#), [31](#), [32](#), [34](#), [38](#), [42](#), [43](#), [45](#).
gam: [42](#).
gmask: [34](#), [35](#), [38](#).
good: [32](#), [42](#), [45](#).
group: [28](#), [32](#), [33](#), [34](#), [45](#).
groups: [34](#), [38](#).
has: [19](#), [27](#), [40](#).
hint: [31](#).
i: [6](#), [7](#), [9](#), [14](#), [16](#), [28](#), [31](#), [36](#), [40](#), [42](#), [44](#).
i0: [6](#).
i1: [6](#).
in: [29](#), [33](#).
init: [9](#).
inner: [41](#).
inside: [36](#), [38](#).
Int: [2](#).
int: [6](#), [7](#), [8](#), [9](#), [10](#), [14](#), [17](#), [19](#), [20](#), [21](#), [26](#), [28](#), [29](#), [31](#), [32](#), [33](#), [34](#), [36](#), [37](#), [39](#), [41](#), [42](#), [44](#), [45](#).
j: [6](#), [7](#), [9](#), [14](#), [16](#), [22](#), [28](#), [31](#), [42](#), [44](#).
j0: [6](#).
j1: [6](#).
k: [12](#), [25](#), [26](#), [34](#), [35](#), [37](#), [38](#), [39](#), [41](#).
keep: [14](#), [16](#), [17](#), [45](#).
keepCell: [29](#).
known: [25](#).
last: [24](#), [35](#).
lb: [22](#).
left: [36](#), [37](#).
len: [20](#), [24](#), [26](#), [32](#), [36](#), [45](#).
limit: [10](#), [12](#), [13](#).
list: [2](#).
m: [12](#), [14](#), [23](#).
main: [1](#).
make: [6](#), [20](#).
mask: [12](#), [13](#).
maxSet: [34](#).
maxSize: [20](#), [26](#).

meets: 19, 35, 36, 39.
Millisecond: 20, 34.
minimalSets: 20, 26, 34.
mode: 2, 3.
n: 9, 10, 12, 13, 14, 19, 27, 29, 33.
n0: 20, 24.
name: 16, 26, 30, 34.
nd: 21, 22, 23.
notMinimal: 26, 27.
Now: 20, 34.
ok: 31, 39, 42, 43.
out: 14, 19, 28, 36, 37, 41, 45.
outer: 41.
p: 9, 10, 11, 12, 13, 14, 16, 27, 29, 40, 41.
Parse: 2.
perms: 42.
pi: 41, 44.
pick: 36, 39.
popcount: 12, 14.
Print: 40, 45.
printed: 44.
Printf: 7, 16, 17, 20, 26, 30, 31, 32, 33, 34, 43, 44, 45.
Println: 3, 26, 33.
puz28a: 3, 15.
puz28b: 3, 15, 45.
puzzle: 8, 9, 10, 14, 27, 29, 40.
puzzles: 34, 38, 40.
q: 27, 45.
r: 7, 42, 43, 45.
rec: 10, 13, 21, 23, 37, 39.
reportSets: 3, 26.
rest: 17.
rho: 42.
Round: 20, 34.
row: 6, 10, 11, 12, 13, 21, 23.
rows: 14, 16.
s: 9, 19, 24, 26, 27, 35, 36, 38, 39, 42, 43.
save: 39.
search: 3, 34.
seen: 32, 45.
set: 19, 23, 29, 34, 36, 45.
sets: 26, 34, 35, 38.
share: 38.
shares: 37, 38.
show: 20.
showSquares: 3, 7.
sig: 42.

Since: 20, 34.
size: 2, 3, 19, 20, 21, 22, 23, 36.
spare: 17.
start: 36.
String: 2, 9, 40, 45.
string: 5, 6, 7, 9, 14, 15, 16, 20, 26, 29, 30, 34, 42.
t: 19, 36.
t0: 20, 34.
three: 41.
time: 20, 34.
to: 42.
twoAreNeeded: 29, 31, 32, 45.
uint: 19.
uint16: 10, 11, 12, 13, 14, 21, 23.
uint64: 18.
usable: 36, 38.
v: 6, 23, 37.
valid: 7.
verbose: 2, 4, 40.
w: 7, 19.
want: 34, 38.
which: 2, 3, 6, 16.
x: 41, 44, 45.
x0: 41.
x1: 41.

- ⟨ Carry the groups and the puzzle across 45 ⟩ Used in section 43
- ⟨ Check the formula exactly as answer 43 prints it 44 ⟩ Used in section 43
- ⟨ Check the nine cells the hint names 31 ⟩ Used in section 30
- ⟨ Check the nine groups that partition the square 32 ⟩ Used in section 30
- ⟨ Count the clues in each group 33 ⟩ Used in section 16
- ⟨ Count the clues that could be dropped 17 ⟩ Used in section 16
- ⟨ Declarations 4 ⟩ Used in section 1
- ⟨ Do what the mode asks 3 ⟩ Used in section 1
- ⟨ File each set under the last group it touches 35 ⟩ Used in section 34
- ⟨ File the new sets under their last cell 24 ⟩ Used in section 20
- ⟨ Functions 6 ⟩ Used in section 1
- ⟨ Give up if the columns already promise too much 22 ⟩ Used in section 21
- ⟨ Look for the minimal sets of this size 21 ⟩ Used in section 20
- ⟨ Note the digits the clues use up 11 ⟩ Used in section 10
- ⟨ Pick the cell with fewest candidates, or report a solution 12 ⟩ Used in section 10
- ⟨ Read the command line 2 ⟩ Used in section 1
- ⟨ Run over every way of placing the clues 38 ⟩ Used in section 34
- ⟨ Test this candidate for good 40 ⟩ Used in section 39
- ⟨ Test this set the slow, honest way 27 ⟩ Used in section 26
- ⟨ Try each candidate there 13 ⟩ Used in section 10
- ⟨ Try every digit that fits in this cell 23 ⟩ Used in section 21
- ⟨ Walk the groups, pruning as each one is settled 39 ⟩ Used in section 38

Eighteen Clues

	Section	Page
Introduction	1	1
The three squares	5	3
Filling in a sudoku	8	6
The two printed puzzles	15	10
Unavoidable sets	18	11
The nine groups	28	16
Every puzzle of a given size	34	19
The isotopism	41	23
Index	46	26